Handling Permanent Failures via Monitoring

If a job has failed and will never succeed, it stands to reason that the only way to fix the underlying problem is first to become aware of it. An extremely common way to do this is to configure an *error catching service* (or *error catcher*), like Honeybadger[7] or Bugsnag.[8]

An error catcher is a service that integrates with your app to receive notifications of any unhandled exception, including those from your Sidekiq jobs. The error catcher then notifies you about the exception via email or an alerting system like Pager Duty. The idea is that you don't have to proactively check for failed jobs; instead, you allow the error catcher to notify you if a job fails. The example app you set up includes a mock error catcher so you can see how it works without having to sign up for a real service and manage that integration.

To connect Sidekiq to an error catcher, you'll need to configure an *error handler*. Sidekiq's configuration provides the attribute `error_handlers`, an array of `Proc` objects that are called when an unhandled error is caught. Our mock error catcher has a client library class included in the example app `ErrorCatcherServiceWrapper`, which has a method named `notify` that will send the exception to the service.

To set this up, you'll need to create an initializer for Sidekiq in `config/initializers/sidekiq.rb`. It will look like the following code, with the `Proc` configured for the error catcher:

snapshots/2-1/sidekiq-book/config/initializers/sidekiq.rb

```
Sidekiq.configure_server do |config|
  config.error_handlers << ->(exception,context_hash) {
    ErrorCatcherServiceWrapper.new.notify(exception)
  }
end
```